

BEGINNER'S GUIDE

Linux

A complete, friendly introduction — from your first login to writing your first shell script.

12	100+	0
Chapters	Commands	Prior experience needed

Table of Contents

1 **What is Linux?** — History, kernel vs. OS, why it matters

2 **Choosing a Distribution** — Ubuntu, Fedora, Debian, Mint & more

3 **Getting Started** — Installing, the desktop, the terminal

4 **The File System** — Directory structure & navigation

5 **Working with Files** — Create, copy, move, delete, view

6 **Permissions & Ownership** — chmod, chown, users & groups

7 **Package Management** — apt, dnf, pacman, snap, flatpak

8 **Processes & System Monitoring** — ps, top, kill, systemctl

9 **Text Editors** — nano, vim basics

10 **Networking Basics** — ping, ssh, curl, ip

11 **Shell Scripting Basics** — Variables, loops, your first script

12 **Cheat Sheet** — Everything on one page

CHAPTER 01

What is Linux?

Understanding the kernel, the OS, and why millions of developers love it

Linux is a free, open-source **operating system kernel** originally created by Finnish student **Linus Torvalds** in 1991. Technically, "Linux" refers only to the kernel — the core program that manages your computer's hardware (CPU, memory, disks). What most people call "Linux" is really a full operating system built around that kernel, usually paired with GNU tools, so it's often called **GNU/Linux**.

Why it matters: Linux powers the vast majority of the world's servers, all Android phones, most cloud infrastructure (AWS, Google Cloud), supercomputers, and embedded devices like routers and smart TVs. Learning it is one of the highest-leverage skills in tech.

Key Characteristics

- **Free & Open Source** — the source code is public; anyone can inspect, modify, and redistribute it.
- **Multi-user & multi-tasking** — built from the ground up to handle many users and processes at once.
- **Stable & secure** — rarely needs reboots; strong permission model.
- **Highly customizable** — from the desktop environment to the kernel itself.
- **Huge community** — forums, wikis, and package repositories for nearly everything.
- **Free to use** — no license fees, ever.

Linux vs. Windows vs. macOS

Aspect	Linux	Windows	macOS
Cost	Free	Paid license	Bundled with Apple hardware
Source code	Open source	Closed source	Mostly closed (Darwin core is open)
Customization	Extremely high	Limited	Limited
Software installs	Package managers (apt, dnf...)	.exe installers, Store	App Store, .dmg
Typical use	Servers, developers, embedded	General desktop, gaming	Creative & general desktop

Good to know: You don't have to "switch" completely. You can try Linux risk-free in a virtual machine (VirtualBox, VMware) or from a bootable USB drive before installing it on real hardware.

CHAPTER 02

Choosing a Distribution

A "distro" packages the Linux kernel with software, a desktop, and a package manager

A Linux **distribution** (distro) bundles the kernel with system utilities, a package manager, and often a graphical desktop. Beginners often worry about picking the "right" one — the good news is most day-to-day commands are identical across all of them.

Popular Beginner-Friendly Distros

MOST POPULAR
Ubuntu
Beginner-friendly, huge community, great hardware support. Based on Debian. Uses `apt`.

VERY FAMILIAR
Linux Mint
Windows-like desktop out of the box, extremely stable. Based on Ubuntu.

ROCK SOLID
Debian
The stable foundation many other distros build on. Great for servers.

CUTTING EDGE
Fedora
Newer software, backed by Red Hat. Uses `dnf`.

ENTERPRISE
RHEL / CentOS Stream
Common in corporate servers. Uses `dnf` / `yum`.

DIY / ADVANCED
Arch Linux
Minimal, you build it up yourself. Uses `pacman`. Steep learning curve.

Which One Should You Pick?

You want...	Try this distro
The easiest possible start	Linux Mint or Ubuntu
Something that feels like Windows	Linux Mint (Cinnamon edition)
The newest apps & features	Fedora
To learn Linux deeply, hands-on	Arch Linux (advanced)
To run a server	Ubuntu Server or Debian

Beginner tip: Start with Ubuntu or Linux Mint. Once you're comfortable with the terminal and general concepts, experimenting with other distros becomes much easier.

CHAPTER 03

Getting Started

Installing Linux and meeting the terminal for the first time

Ways to Try Linux

Live USB Boot from a USB stick without installing anything. Nothing is changed on your hard drive.

Virtual Machine Run Linux inside a window on your current OS using VirtualBox or VMware — completely safe.

Dual Boot Install Linux alongside Windows/macOS; choose which one to boot into at startup.

WSL (Windows) "Windows Subsystem for Linux" runs a real Linux environment inside Windows.

Meet the Terminal

The **terminal** (also called the shell or command line) is a text-based way to interact with the operating system. It looks intimidating at first, but it's simply typing commands and reading the responses. The default shell on most distros is **bash** (some newer ones use **zsh**).

```
user@ubuntu:~$ whoami
user
user@ubuntu:~$ pwd
/home/user
user@ubuntu:~$ date
Wed Sep  9 10:32:04 IST 2026
```

Reading the prompt: `user@ubuntu:~$` means user **"user"**, on machine **"ubuntu"**, currently in directory **~** (home). The `$` means you're a regular user; a `#` would mean you're root (administrator).

Your First Commands

Command	What it does
`whoami`	Shows your current username
`pwd`	Prints working directory (where you are)
`date`	Shows current date and time
`clear`	Clears the terminal screen
`man <command>`	Opens the manual page for a command
`exit`	Closes the terminal session

> **Tip:** Stuck on a command? Type `man ls` (or `ls --help`) and the built-in manual will explain every option. Press `q` to quit the manual viewer.

CHAPTER 04

The File System

How Linux organizes everything — and how to move around in it

Unlike Windows (which has separate drive letters like `C:`), Linux has a single unified tree starting at the **root directory**, written as `/`. Everything — files, folders, and even devices — lives somewhere under it.

Key Directories

Path	Purpose
`/`	Root of the entire filesystem
`/home`	Personal folders for each user (e.g. `/home/alex`)
`/etc`	System-wide configuration files
`/bin`, `/usr/bin`	Essential program executables
`/var`	Variable data: logs, caches, mail
`/tmp`	Temporary files, cleared on reboot
`/root`	Home directory of the root (admin) user
`/dev`	Device files (disks, USB, etc.)
`/mnt`, `/media`	Mount points for external/removable drives
`/opt`	Optional third-party software

Navigating with the Terminal

```
# Show where you are
pwd

# List files in the current directory
ls
ls -l      # long format, with details
ls -la     # include hidden files (start with .)

# Change directory
cd Documents      # go into Documents
cd ..             # go up one level
cd ~              # go to home directory
cd /              # go to root directory
cd -              # go to previous directory
```

Absolute vs. Relative Paths

Absolute path — starts from root `/`, always points to the same place regardless of where you are.

```
cd /home/alex/Documents
```

Relative path — starts from your current location.

```
cd Documents/reports
```

> **Shortcuts:** `.` means current directory, `..` means parent directory, `~` means your home directory, and `-` means the previous directory you were in.

CHAPTER 05

Working with Files

Create, view, copy, move, and delete — the core daily workflow

Creating Files & Folders

```
touch notes.txt          # create an empty file
mkdir projects            # create a folder
mkdir -p a/b/c             # create nested folders at once
```

Viewing File Contents

Command	Use case
`cat file.txt`	Print the whole file to the screen
`less file.txt`	Scroll through a large file page by page
`head file.txt`	Show the first 10 lines
`tail file.txt`	Show the last 10 lines
`tail -f log.txt`	Follow a file live as it grows (e.g. logs)

Copy, Move, Rename, Delete

```
cp file.txt backup.txt        # copy a file
cp -r folder1 folder2          # copy a folder recursively
mv file.txt newname.txt        # rename (mv is also used to move!)
mv file.txt ~/Documents/        # move into another folder
rm file.txt                     # delete a file
rm -r folder/                   # delete a folder and its contents
```

Careful: `rm` does **not** move things to a recycle bin — deletion is immediate and permanent. Double-check paths before pressing Enter, especially with `rm -rf` (force, recursive), which is one of the most dangerous commands in Linux.

Finding Files & Text

```
find /home -name "*.pdf"        # find files by name pattern
find . -type d -name "src"       # find directories named "src"
grep "error" logfile.txt          # search for text inside a file
grep -r "TODO" ./project           # search recursively through folders
locate filename.txt                 # fast search using a pre-built index
```

Wildcards

Wildcard	Meaning	Example
`*`	Any number of characters	`*.txt` → all .txt files
`?`	Exactly one character	`file?.txt` → file1.txt, fileA.txt
`[abc]`	Any one character listed	`file[12].txt` → file1.txt or file2.txt

CHAPTER 06

Permissions & Ownership

Linux's security model: who can read, write, and execute what

Every file and folder has an **owner**, a **group**, and a set of permissions for three categories of people: the owner, the group, and everyone else.

```
$ ls -l notes.txt
-rw-r--r-- 1 alex staff  1024 Sep  9 10:00 notes.txt
```

Reading the Permission String

The 10 characters break down as:

- **Position 1**: file type (`-` file, `d` directory, `l` symlink)
- **Positions 2–4**: owner's permissions
- **Positions 5–7**: group's permissions
- **Positions 8–10**: everyone else's permissions

Each triplet is `r` (read), `w` (write), `x` (execute) — or `-` if not granted.

`rw-r--r--` = owner can read/write, group can read, others can read.

Changing Permissions with chmod

Permissions can be set with letters (symbolic) or numbers (octal): `r=4, w=2, x=1`, added together per category.

```
chmod u+x script.sh        # add execute permission for the owner
chmod go-w file.txt         # remove write permission for group & others
chmod 755 script.sh          # owner: rwx (7), group: r-x (5), others: r-x (5)
chmod 644 file.txt            # owner: rw- (6), group: r-- (4), others: r-- (4)
```

Common Octal Combinations

Number	Permission
7	rwx — read, write, execute
6	rw- — read, write
5	r-x — read, execute
4	r-- — read only
0	--- — no access

Ownership & the Superuser

```
sudo chown alex file.txt         # change owner
sudo chown alex:staff file.txt    # change owner and group
sudo chgrp staff file.txt          # change group only
```

What is sudo? `sudo` ("superuser do") temporarily runs a single command with administrator (root) privileges. You'll be asked for your password. Use it deliberately — running everything as root is a common beginner mistake that can damage your system.

Users & Groups

```
whoami                    # current user
id                          # your UID, GID, and groups
sudo adduser newuser        # create a new user (Debian/Ubuntu)
sudo usermod -aG sudo alex   # add alex to the sudo group
groups alex                   # show which groups alex belongs to
```

CHAPTER 07

Package Management

Installing, updating, and removing software the Linux way

Instead of downloading installers from random websites, Linux distros use **package managers** that fetch trusted software from central repositories, handle dependencies automatically, and keep everything updated with one command.

By Distribution Family

Family	Distros	Manager	Package format
Debian-based	Ubuntu, Mint, Debian	`apt`	.deb
Red Hat-based	Fedora, RHEL, CentOS	`dnf` (was `yum`)	.rpm
Arch-based	Arch, Manjaro	`pacman`	pkg.tar.zst
SUSE-based	openSUSE	`zypper`	.rpm

apt (Ubuntu / Debian / Mint)

```
sudo apt update                 # refresh the list of available packages
sudo apt upgrade                 # upgrade all installed packages
sudo apt install htop             # install a package
sudo apt remove htop               # remove a package
sudo apt autoremove                 # clean up unused dependencies
apt search "text editor"             # search for a package
```

dnf (Fedora / RHEL)

```
sudo dnf check-update
sudo dnf upgrade
sudo dnf install htop
sudo dnf remove htop
```

pacman (Arch)

```
sudo pacman -Syu          # sync repos + full system upgrade
sudo pacman -S htop         # install
sudo pacman -R htop          # remove
```

Universal Formats

Snap (Canonical) — sandboxed, auto-updating apps that work across distros.

```
sudo snap install spotify
```

Flatpak — similar concept, very popular on Fedora/GNOME desktops.

```
flatpak install flathub org.videolan.VLC
```

> **Tip:** Always prefer your distro's official repositories or Flatpak/Snap over downloading random binaries from the web — it's safer and keeps everything updated in one place.

CHAPTER 08

Processes & System Monitoring

Seeing what's running, using resources, and how to stop it

Viewing Processes

```
ps aux                # snapshot of all running processes
ps aux | grep firefox     # find a specific process
top                          # live, auto-refreshing process viewer
htop                           # nicer, colorful version of top (may need install)
```

Reading `top` / `htop`

Column	Meaning
PID	Process ID — a unique number for each running process
%CPU	Percentage of CPU currently used
%MEM	Percentage of RAM currently used
USER	Who owns/started the process
COMMAND	The program name or command line

Stopping Processes

```
kill 1234                # politely ask process 1234 to terminate
kill -9 1234               # force-kill it immediately
killall firefox              # kill every process matching a name
pkill -f "node server.js"     # kill by matching command text
```

Background & Foreground Jobs

```
long_task.sh &      # run in the background
jobs                  # list background jobs
fg %1                  # bring job 1 to the foreground
Ctrl + Z                # suspend the current foreground job
bg %1                     # resume a suspended job in the background
```

Managing Services with systemctl

```
sudo systemctl status nginx      # check if a service is running
sudo systemctl start nginx        # start it
sudo systemctl stop nginx          # stop it
sudo systemctl restart nginx        # restart it
sudo systemctl enable nginx          # start automatically on boot
```

Disk & Memory Usage

```
df -h          # disk space, human-readable
du -sh folder/   # size of a specific folder
free -h            # RAM and swap usage
```

CHAPTER 09

Text Editors

Editing configuration files and scripts straight from the terminal

nano — The Friendly Beginner Editor

Nano shows its shortcuts at the bottom of the screen at all times, making it the easiest place to start.

```
nano notes.txt
```

Shortcut	Action
`Ctrl + O`	Save (write out)
`Ctrl + X`	Exit
`Ctrl + K`	Cut the current line
`Ctrl + U`	Paste
`Ctrl + W`	Search for text

vim — Powerful but Modal

Vim has separate **modes**: Normal mode (for navigation/commands) and Insert mode (for typing text). This trips up beginners, so here's the essential survival kit:

```
vim notes.txt
```

Key(s)	Action
`i`	Enter Insert mode (start typing)
`Esc`	Return to Normal mode
`:w`	Save
`:q`	Quit
`:wq`	Save and quit
`:q!`	Quit without saving (discard changes)
`dd`	Delete the current line
`/word`	Search for "word"

> **Famous joke:** "How do I exit vim?" is one of the most-searched Linux questions ever. Remember: `Esc` then `:q!` gets you out without saving.

GUI Alternatives

If you're on a desktop environment, graphical editors like **gedit**, **Kate**, **VS Code**, or **Sublime Text** work just like Notepad — no terminal required.

CHAPTER 10

Networking Basics

Checking connectivity, transferring files, and remote access

Checking Connectivity

```
ping google.com             # test connectivity (Ctrl+C to stop)
ip a                           # show network interfaces & IP addresses
ip route                         # show routing table / default gateway
hostname -I                        # quick view of your local IP
```

DNS & Connectivity Tools

```
nslookup example.com
dig example.com
curl -I https://example.com      # fetch just the HTTP headers
wget https://example.com/file.zip  # download a file
```

Remote Access with SSH

```
ssh user@192.168.1.10          # connect to a remote machine
ssh -p 2222 user@host            # connect on a custom port
scp file.txt user@host:/path/     # securely copy a file to a remote machine
ssh-keygen -t ed25519               # generate an SSH key pair
ssh-copy-id user@host                # install your public key on a remote server
```

> **SSH keys > passwords:** Once you copy your public key to a server, you can log in without typing a password every time — and it's far more secure.

Firewall Basics (ufw — Ubuntu)

```
sudo ufw status
sudo ufw allow 22       # allow SSH
sudo ufw allow 80/tcp     # allow HTTP
sudo ufw enable
```

CHAPTER 11

Shell Scripting Basics

Automate repetitive tasks by chaining commands into a script

A shell script is simply a text file full of the same commands you'd type manually, run all at once.

Your First Script

```
#!/bin/bash
# This is a comment
echo "Hello, $(whoami)! Today is $(date +%A)."
```

```
chmod +x hello.sh    # make it executable
./hello.sh              # run it
```

Variables

```
name="Alex"
echo "Hello, $name"

# Reading user input
read -p "Enter your name: " name
echo "Nice to meet you, $name"
```

Conditionals

```
if [ -f "file.txt" ]; then
  echo "File exists"
else
  echo "File not found"
fi
```

Loops

```
# for loop
for i in 1 2 3 4 5; do
  echo "Number: $i"
done

# while loop
count=1
while [ $count -le 5 ]; do
  echo "Count: $count"
  count=$((count + 1))
done
```

Useful Operators & Piping

Symbol	Meaning
`|`	Pipe — send output of one command into another
`>`	Redirect output to a file (overwrite)
`>>`	Redirect output to a file (append)
`&&`	Run next command only if the previous succeeded
`;`	Run commands sequentially regardless of success

```
ls -l | grep ".txt"                  # pipe: list files, filter for .txt
echo "log entry" >> activity.log       # append to a log file
mkdir backup && cp *.txt backup/           # only copy if mkdir succeeded
```

Practice idea: Write a script that backs up your Documents folder into a dated zip file, then schedule it to run daily using `cron` (`crontab -e`).

CHAPTER 12

Cheat Sheet

The essentials, all in one place for quick reference

Navigation

`pwd`	current directory
`ls -la`	list all files
`cd path`	change directory
`cd ..`	go up one level

Files & Folders

`touch f`	create empty file
`mkdir d`	create folder
`cp a b`	copy
`mv a b`	move / rename
`rm f`	delete file
`rm -r d`	delete folder

Viewing Content

`cat f`	print file
`less f`	page through file
`head/tail f`	first/last lines
`grep x f`	search in file

Permissions

`chmod 755 f`	set permissions
`chown u:g f`	change owner/group
`sudo cmd`	run as admin

Packages (apt)

`apt update`	refresh lists
`apt upgrade`	upgrade all
`apt install x`	install package
`apt remove x`	remove package

Processes

`ps aux`	list processes
`top / htop`	live monitor
`kill -9 pid`	force stop
`systemctl status x`	service status

Networking

`ping host`	test connection
`ip a`	show IP addresses
`ssh user@host`	remote login
`scp a user@host:/p`	copy over SSH

Disk & Memory

`df -h`	disk space
`du -sh d`	folder size
`free -h`	RAM usage

Keyboard Shortcuts in the Terminal

Shortcut	Action
`Ctrl + C`	Cancel the current command
`Ctrl + Z`	Suspend the current process
`Ctrl + D`	Exit the shell / send EOF
`Ctrl + L`	Clear the screen (like `clear`)
`Ctrl + R`	Search previous commands (history)
`Tab`	Auto-complete file/command names
`↑ / ↓`	Cycle through command history

You've now covered the core of everyday Linux use. The best next step is simply practicing — open a terminal and try these commands on real files. Happy hacking! 🐧